

Reporting Ceph issues from the Ceph-Dashboard

13th April 2021

Personal Information

Name: Shreyaa Sharma

Location: New Delhi, India

Time zone: UTC+05:30 (Indian Standard Time)

Education: B.Tech Civil Engineering, Indian Institute of Technology Roorkee (Batch of 2023)

Email ID: `shreyasharma.ss305@gmail.com`

Alternate Email ID: cypherean@gmail.com

Github: [cypherean](https://github.com/cypherean)

About Me

I am a sophomore at IIT Roorkee currently pursuing B.Tech. in Civil Engineering. I have always loved the concept of open source and wanted to contribute to the same. I have been trying to contribute in the small ways I could, by having all my projects on GitHub so that others can benefit from the same, trying to fix minor bugs, and participating in various open-source programs. GSoC provides a great opportunity to work in open-source projects with the core developers. This makes me excited and eager to participate in it.

I have been interested in software development since my freshman year and have been exploring this space since then. I am a developer at SDS Labs which fosters and promotes technical activities on the campus. It caters to all the technical needs of the students. Under the guidance of the group, I've worked on various open-source projects.

OVERVIEW

When a Ceph user encounters an issue, they have to gather the information from the command-line interface and manually open an issue in the Ceph defect tracker. Sometimes, this information is too large and the process tedious. This project aims to provide Ceph users with the ability to report issues directly from the Ceph-Dashboard UI and attach the minimal information for opening an issue while the widget takes necessary information to reproduce the issue on user's approval automatically. The main reason for adding the option to report issues from the Dashboard is to make it convenient for the users and to automate the process to some extent by auto-filling the details.

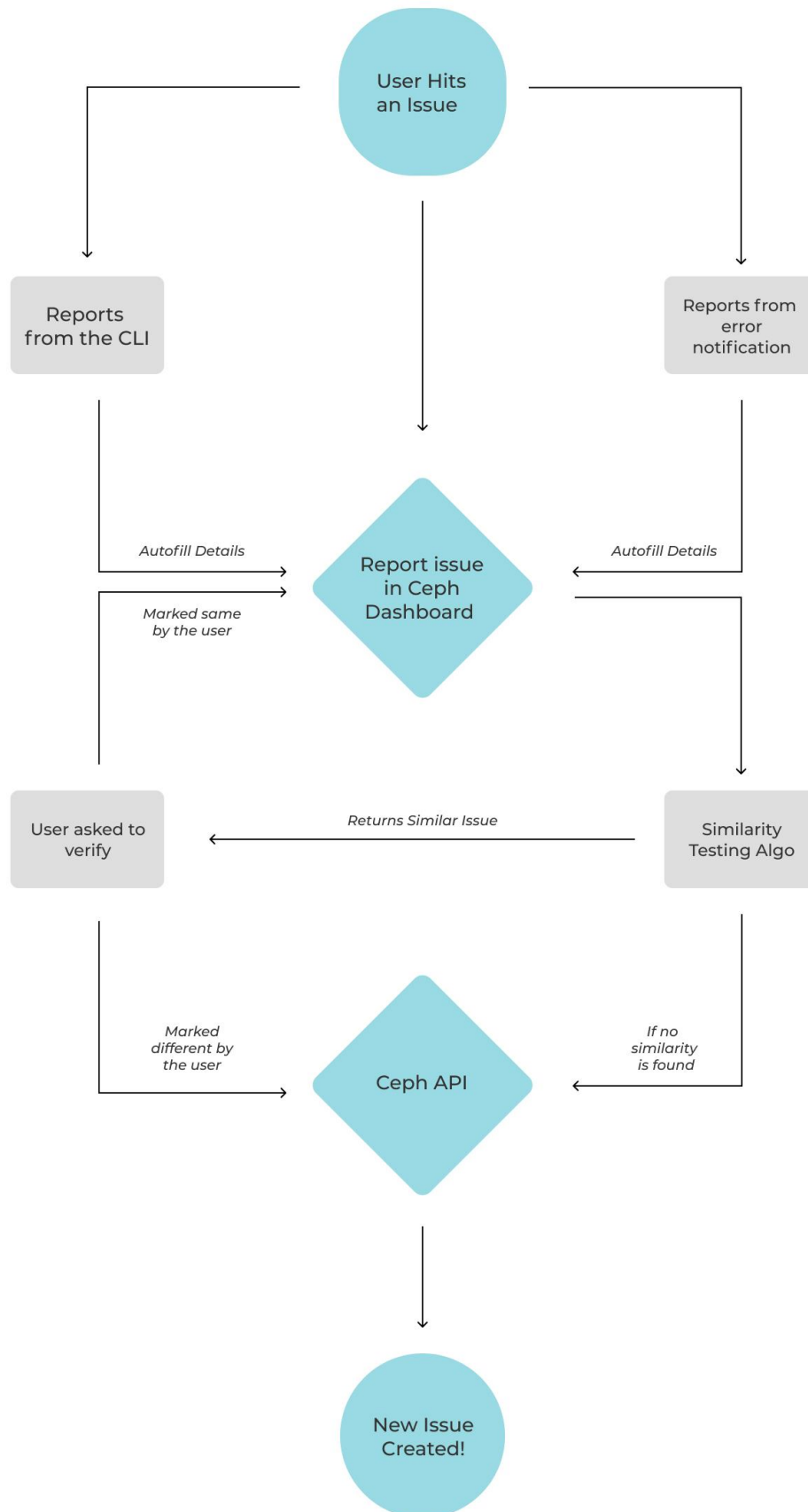
Another purpose this project would serve is reducing the duplication of issues since it checks for similar existing issues before creating a new one and asks the user to check if the issue has already been reported by providing a list of similar issues.

GOALS

1. To add an interface in the Ceph-Dashboard to report issues faced by Ceph users.
2. To provide a link in the CLI to report an issue in case of fatal issues and add a direct button in the error logs/error notifications on Dashboard which redirects to the issue report tab.
3. To collect system and development information, and error logs automatically (on user's approval).
4. To test reported issues for similarity with existing issues in both the Dashboard as well as the Ceph Issue Tracker.
5. To integrate Redmine API and python plugins to create new issues from the dashboard itself.
6. Stretch Goal: Increase priority of an issue if it is reported by multiple users.

IMPLEMENTATION DETAILS

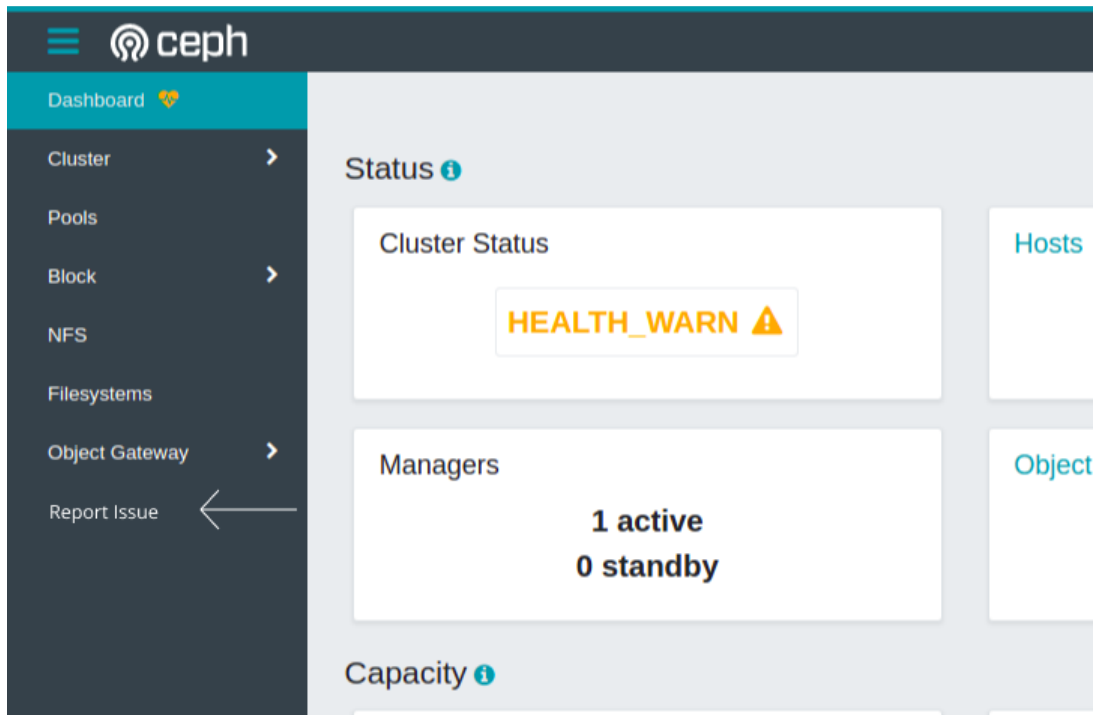
The user should be able to report issues from the Ceph-Dashboard UI. We need a form to take the minimum information required to open an issue in the Ceph issue tracker and automate the process to the maximum extent possible by taking the system and development environment and command logs.



Ceph-Dashboard Report Issue UI

Understanding in terms of the flow chart given above, when a user hits an issue they will have three ways to report it -

1. The Report Issue button in the navigation bar of Ceph-Dashboard can be used to report a new issue.



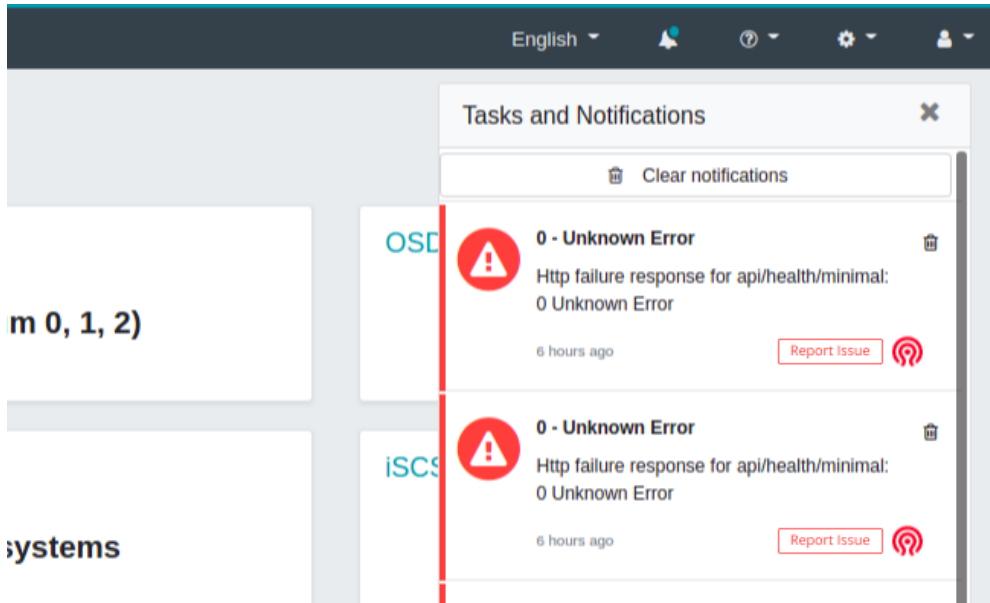
Mock UI of the dashboard after the addition of the “Report Issue” button in the navbar

1. A link in the CLI when a fatal error is encountered. When they click on the link, it'll open the Ceph-Dashboard Report Issue page along with the logs of error.

```
[➔ ~ ceph orch set backend rook]
.
.
.
ERR: Fatal error
If you think this is an issue with Ceph, you can report it here: https://localhost:11000/#/issues?log=temp-12776528.log
➔ ~
```

Mock UI of the CLI issue link on encountering a fatal error

2. A button in the error notification on Ceph Dashboard in case of fatal errors.



Mock UI of the error notifications with an option to report the issue

All of these three ways will take the user to a form page where they can select the issue tracker, give the title and describe the issue and steps to reproduce it. The user has the choice to permit the collection of data that is vital to the issue's reproduction such as the development environment and steps that led to it, in the form of command logs. The logic on how the issue would be reported on the main Ceph Issue Tracker is explained in the subsequent subsections.

A mock UI of a 'Report Issue' form in a dashboard. The form is titled 'Report Issue' and has a sidebar on the left with navigation links: 'Dashboard', 'Cluster', 'Pools', 'Block', 'NFS', 'Filesystems', 'Object Gateway', and 'Report Issue' (highlighted). The form fields include: 'Tracker*' (a dropdown menu), 'Subject*' (a text input), and 'Description*' (a large text area with a placeholder 'Share the steps to reproduce the issue. What did you see that you didn't expect to see?'). Below the text area are three checkboxes: 'Include system information (show)', 'Include server details (show)', and 'Include command logs (hide)'. A light blue box contains a log snippet: 'ceph mgr module enable rook', 'ceph orch set backend rook', 'ceph orch status', and 'Error ENODENT: No orchestrator configured (try "ceph orch set backend")'. Below this box is an 'EDIT' link. At the bottom of the form is a 'Submit Issue' button.

Mock UI of a simplistic Issue Opening Form in the Dashboard

The image shows a mock UI for a Ceph Issue Tracker. On the left is a dark sidebar with a 'ceph' logo and a menu containing: Dashboard (with a heart icon), Cluster, Pools, Block, NFS, Filesystems, Object Gateway, and Report Issue (highlighted in teal). The main content area is titled 'Report Issue' and contains the following fields and elements:

- Tracker***: A dropdown menu with 'Select' and a downward arrow.
- Subject***: A text input field with the placeholder text 'Subject'.
- Description***: A large text area with the placeholder text 'Share the steps to reproduce the issue. What did you see that you didn't expect to see?'.
- Checkboxes**: Three checked checkboxes with labels and links:
 - ☒ Include system information ([show](#))
 - ☒ Include server details ([show](#))
 - ☒ Include command logs ([hide](#))
- Code Snippet**: A light blue box containing the following text:

```
ceph mgr module enable rook
ceph orch set backend rook
ceph orch status
Error ENOENT: No orchestrator configured (try `ceph orch set backend`)
```

Below the code is a teal **EDIT** link.
- Submit Issue**: A teal button at the bottom.

Enlarged Mock UI of a simplistic Issue Opening Form in the Dashboard

After the user adds all the required information in the form and submits it, the data would be analysed for similarity with existing issues.

The issue raised by the user would be checked for duplicates for similarity with existing issues, the user would then be asked to check out the existing similar issues if similarity exists. This is explained in the next subsection.

Tackling Issue Duplicacy

1. Using [Redmine did-you-mean plugin](#) on the Ceph Issue Tracker

One way to deal with similar issues on the Ceph Issue Tracker is to check for similarity in the issue subject. This is done by Redmine's did-you-mean plugin. When users fill in the subject field of new issues, the plugin automatically searches among the existing ones for possible duplicates (i.e. issues having a similar title) and displays them right below the subject field.

This is one way to deduplicate issues on the Ceph Issue Tracker. To integrate this feature in the Ceph Dashboard UI, the existing plugin needs to be modified.

2. Modify Redmine did-you-mean plugin to make it compatible with Ceph-Dashboard

The existing Redmine plugin, did-you-mean can be efficiently used to deduplicate issues at the Ceph Issue Tracker. To integrate this feature on the Ceph-Dashboard which uses python libraries to make use of existing APIs, this plugin needs to be modified. I'll fork this plugin and make necessary changes to make it compatible with the Ceph-Dashboard.

The similar issues would then be listed below the current issue, and the user would be asked to verify for similarity or dissimilarity.

This needs further research before implementation.

The next step is to report the issue in the Ceph tracker using a Ceph issue tracker API.

Interacting with Ceph Issue Tracker

If the user disagrees for similarity, then an issue would be created on the Ceph issue tracker. The details that were not taken from the user would be given a default value such as that for assignee, priority, and date. Information like the Ceph version used can be analysed from the data taken from the user. And a new issue would be submitted on the Ceph Issue tracker.

Upon submitting the issue on the issue form page, they would be able to open the issue on the Ceph Issue Tracker, which is based on the [Redmine Issue tracker](#), using the [Redmine REST API](#). Login would be needed which would be handled using Redmine OAuth (which is work-in-progress: <https://www.redmine.org/issues/24808>) or by requesting the user to add an **API Access Token** fetched from Ceph Issue Tracker.

The two ways to authenticate on the Redmine Rest API are:

- using your regular login/password via HTTP Basic authentication.
- using your API key

To avoid spam, the best option is to ask the user to get an API Access Token from their Ceph account and add it to their Ceph Dashboard account in a one-time process. When the user submits an issue, this API access token would be used to get API key and send requests.

The approach used for this particular part is subject to further research and discussion with mentors and maintainers.

Testing

For the changes introduced so far, I plan on writing a functional test for the report issue API and similarity testing plugin and E2E UI tests for the frontend widget.

1. *E2E UI testing in cypress*: Since this project makes major changes in the frontend, testing these changes is also just as important. The changes that needed to be tested are:
 - a. Report Issue Button in the navbar: The existence of the button and redirect to the correct link on clicking it needs to be tested.
 - b. Error button in notification and correct redirection on clicking it.
 - c. Report Issue Form: The most important change introduced in this project is the Report Issue Form. The different components of the form need to be tested in the frontend. If visual regression testing is introduced, the complete form can be tested by taking a snapshot since the form is made up of static components that only change on user's actions.
2. *Functional Tests*: The project is divided into several units, hence it becomes necessary to test individual units.
 - a. Redmine API Changes - As we are using an external project as the issue tracker, the project is susceptible to API changes, which can completely break the feature. Tests which checks for the version of the Redmine Issue Tracker and if the endpoints still work on <https://tracker.ceph.com/>. Tests to check if the API Key expired should also be added.
 - b. Leaking of Sensitive Data - Tests that check if any sensitive data is leaked from somewhere is also added. The logs and descriptions should never contain the user's personal data. So tests that deliberately adds personal data to the logs and checks if PII Detection (next subsection) algorithm successfully redacts the personal data.
 - c. Duplication Tests - Tests where hard-coded and known issues are checked whether they are detected as a duplicate issue or not.

Security Concerns

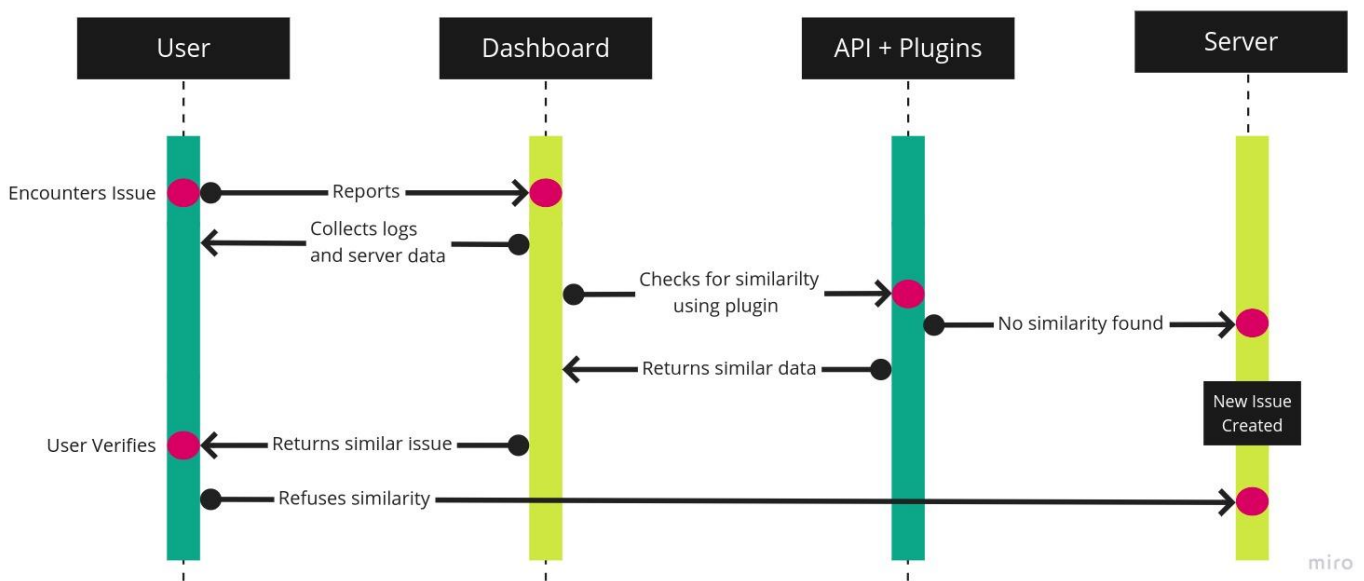
Since the report issue feature would take system and server details, and logged commands there is a huge risk of exposure of sensitive content by the users. Some things to keep in mind while implementing these features are:

1. Redacting local paths, paths only relative to Ceph project root would be taken.
2. PII detection problem is well known and well-studied ML problem. It should be very fast to use and needs to be tweaked for our use case.
3. Stacktraces should be returned but not data entries.
4. The user has the option to choose which information is being sent to the server. He/she can see and edit the data being sent.

Modify priority on duplicate issues (secondary goal)

One way to handle duplicates was to ask the user whether they want to open a new issue or not. A better solution can be to update the priority of the already existing issue. This can be implemented by giving an option for the user to show that *they are also facing the same issue*. Since Redmine doesn't have any way to upvote, so we can use comments. After a fixed number of comments from distinct users, let us say 3 users faced the same problem, the priority of the issue can be increased. This goal is a secondary goal after the basic functionality is completed.

Sequence Diagram:



Sequence Diagram of the project

MILESTONES

Pre GSoC Period (14th April - 16th May)

-
1. Continue to work on existing issues.
 2. Modify an existing API in the Ceph Dashboard.
 3. Make corresponding changes in the front end.

Community Bonding Period (17th May- 6th June)

1. Work closely with the community to design an interface for the form and understand the working of the Ceph Issue tracker.
2. Research on plugins and API suitable for the project.

Coding Period

7th June - 15th June

1. Make final changes to the design of the 'Report Issue' form of Ceph-Dashboard UI.
2. Write the front end for the Ceph Dashboard report issue form.

16th June - 22nd June

1. Isolate fatal errors that could be encountered and need to be reported.
2. Add redirect link in CLI and dashboard error notification to the report issue tab.

23rd June - 30th June

1. Work on collecting system and server details and logged commands automatically.
2. Work on authentication of users to report an issue.

1st July - 7th July

1. Integrate API on the dashboard.
2. Add Redmine plugin to issue tracker.

8th July - 16th July

1. Buffer time to complete existing tasks if any.
2. Prepare for Phase 1 Evaluation

17th July - 31st July

1. Modify the existing 'did-you-mean' plugin for Ceph Dashboard.

1st August - 7th August

1. Add modified plugin in Ceph Dashboard
2. Make corresponding frontend fixes.

7th August - 15th August

1. Write functional and E2E tests.

16th August - 23rd August

1. Buffer time to complete pending tasks, if any.
2. Prepare for final Evaluation Phase

Statement of Motivation

I chose this particular project because I have been highly interested in system integration and servers in the past. I wish to learn more about cloud and servers, and I feel this would be a great opportunity to learn about this field while working with the tech stack that I already have a sound knowledge of.

My skills and work experience align well with the skills needed for this project such as Web development using React, Vue, and Angular, Python, and REST APIs.

I have worked on many front-end projects as a part of SDS Labs, a software development group in my college, and worked on both frontend and backend in group projects developed for hackathons.

I have contributed to some open source projects like Public Lab, Mozilla, and Swift in the past.

Open Source Experience

Public Lab Editor- Outreachy

Last summer, I worked as an Outreachy intern at Public Lab. My project was Fortification of the Editor and Adding Custom inserts. During the internship, I worked on integrating e2e testing in the project, handled a major dependency upgrade and subsequent release, and solved the existing issues that hindered a rich user experience.

Pinger

Pinger is an open-source implementation of the Status Page written in Go. I wrote the alert system for the project. When a service fails or is revived the status changes. Every status change is reported either on slack, discord, or mail depending on the alert configuration of the user.

Ikai

Ikai is a mobile application that can scan, edit multimedia, share files in both online and offline mode and has the chat feature to facilitate smooth remote collaboration. I implemented offline file sharing using TCP and rn-fetch-blob.

CricLive

CricLive takes text cricket commentary as input and generates a cricket match in real-time using Artificial Intelligence and animations.

Kiwi

Kiwi is an in-memory key-value store written in Go. I implemented the function to add JSON support for storing the values in JSON format and using them in Go format from a stored file.

Ceph

I recently started working on an issue in Ceph

- [ceph/ceph#40319 - skip-ssh flag enables cephadm mgr module](#)

Time commitment

I would be easily able to work more than 20 hours a week. The college's summer vacations have been shifted ahead to the entire months of June, July and extending to the first week of August. Therefore, there would be no classes for the first two months of the program's coding period and no exams throughout the coding period. My college would reopen from 4th August. During this time also, I would easily be able to work more than 20 hours. I plan to complete most of the work before my college reopens.

Other than this project, I have no commitments/vacations planned for the summer. I shall keep my status posted to all the community members on a weekly basis and maintain transparency in the project.

Post GSoC

I have learned a lot and was introduced to a completely new ecosystem by contributing to the Ceph and even after the GSoC period ends, I plan on contributing to this organization, by adding to my past projects and working on open issues.

The aforementioned project that I will be working on is not only going to help dashboard users with a better issue reporting experience but also a generated agreement for the CLI users. By integrating the issue tracker in the dashboard itself and automating the information collection, the user experience will be improved. Reducing the duplicate issues makes issue handling and fixing easier for the maintainers.

The highlight of my experience so far with Ceph has been the fast-planned development, the active involvement of the mentors, and the rigorous testing practices adopted in the project. With the community growing continuously, I feel responsible for all the projects I'm a part of. Having picked up a lot of development skills, my major focus would be to help the project and the community grow.